Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación
IIC2233 — Programación Avanzada 2026-1

Publicación: 26 de Marzo de 2026
Actividad evaluada

Actividad 3

Programación Orientada a Objetos Avanzado

Entrega

- **Lugar:** Repositorio personal de GitHub — Carpeta: `Actividades/AC3`
- **Fecha máxima de entrega:** 26 de Marzo 17:20
- **Ejecución de actividad**: La Actividad será ejecutada **únicamente** desde la terminal del computador. **Los *paths* relativos utilizados en la Actividad deben ser coherentes con esta instrucción, y no pueden modificarse.**

Importante: Antes de comenzar, comprueba que Git este funcionando correctamente en tu repositorio privado. Para esto, **sube los archivos base de la actividad de inmediato** (*add*, *commit*, *push*). Se espera que en esta actividad (así como en las demás actividades y tareas) utilices Git a lo largo de **todo tu desarrollo** como una herramienta, no sólo como un método de entrega. Es por esto que recomendamos enfáticamente que vayas subiendo tus cambios constantemente (***push***), ya que **problemas de último minuto** relacionados con la entrega y Git **no serán considerados**.

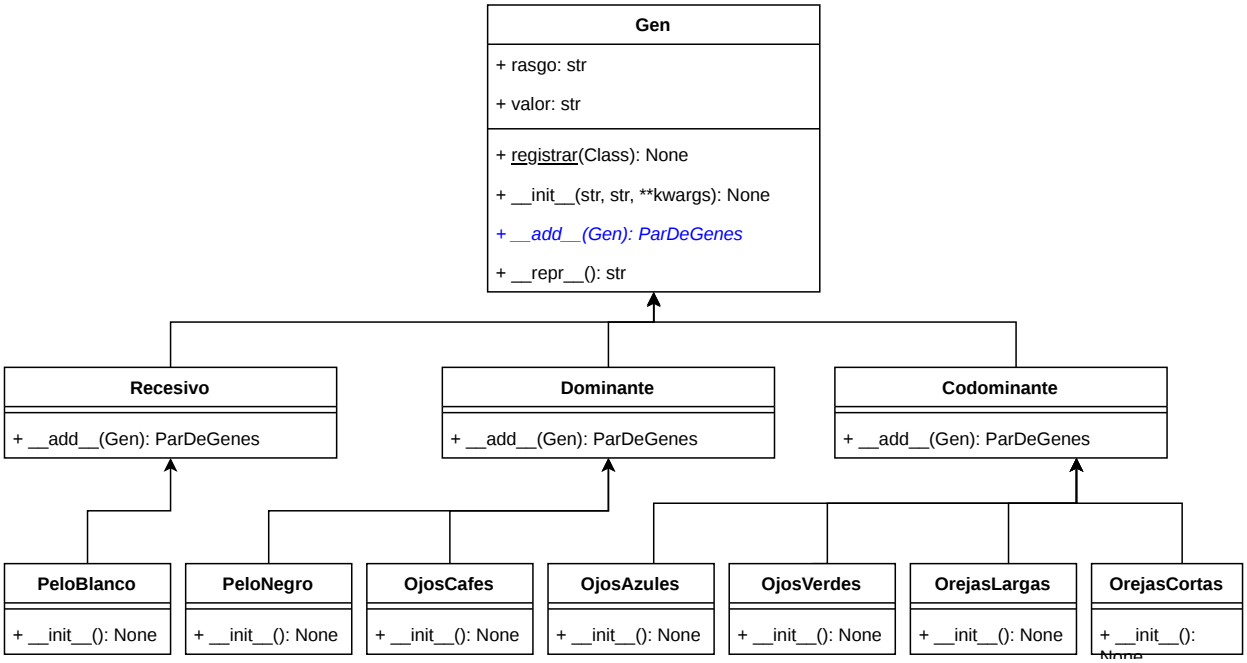
Introducción

En el laboratorio genético de perros, se estudia el comportamiento de ciertos genes de perro como el color del pelo, el largo de las orejas, y el color de sus ojos.

Te pidieron a ti que los ayudes a completar un programa que modela el comportamiento de los genes de perro.

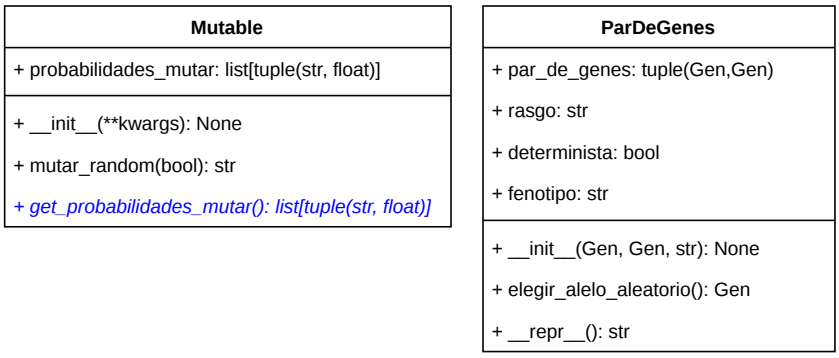
Diseño del programa

El programador anterior, creó un modelo de clases que ayuda a manejar el comportamiento de cada gen:



Esta estructura de clases permite definir el comportamiento general de los genes en la clase abstracta `Gen`; el comportamiento de dominancia de genes en las clases intermedias `Recesivo`, `Dominante` y `Codominante`; y el comportamiento particular de cada gen en las clases de cada gen.

Adicionalmente, el programador anterior creó las clases `ParDeGenes`, `Perro` y `Mutable` para modelar el comportamiento de los genes de a pares en los perros, y la posibilidad de que estos muten:



Archivos

En el directorio de la actividad encontrarás lo siguiente:

- No modificar `bases.py`: Contiene las clases `Gen`, `Mutable` y `ParDeGenes`. No debes modificar nada en este archivo, pero se recomienda que leas el código, ya que las clases que debes modificar heredan de estas.

- Modificar Entregar `dominancia.py`: Contiene las clases `Dominante`, `Recesivo` y `Codominante`, que manejan el comportamiento de 2 genes al estar juntos.

  Trabajarás en este archivo en la Parte 1 de esta actividad.

- Modificar Entregar `caracteristicas.py`: Contiene las clases dedicadas a cada gen en particular. Cada una corresponde a un rasgo físico y un valor para ese rasgo. Por ejemplo la clase `PeloBlanco` tiene `rasgo = pelo` y `valor = Blanco`.

  Trabajarás en este archivo en la Parte 2 de esta actividad.

- No modificar `utils_random.py`: Este archivo contiene métodos aleatorios que permiten elegir qué genes se pasan de un perro a otro y qué genes mutan. No debes modificarlo ya que puede generar problemas al corregir tu actividad.

- Modificar `main.py`: Contiene código que registra las sub-clases de Gen para poder acceder fácilmente a ellas. En este archivo puedes modificar todo lo que hay después de "`if __name__ == "__main__":`" para probar tu código.

- No modificar `tests_publicos/`: Este carpeta contiene tests que comprueban el funcionamiento correcto de tu código usando casos de ejemplo. Puedes usar los tests para probar tu código pero no modifiques nada en esta carpeta.

Modelo de datos

En esta sección se ve en detalle las clases del archivo `bases.py`.

- No modificar `class Gen`:
  - `REGISTRO`: Es una atributo de clase de tipo diccionario que pertenece a la clase Gen. Puede ser accedida usando "`Gen.REGISTRO[rasgo][valor]`" para obtener la sub-clase con el rasgo y valor indicados.
  - `def __init__(self, rasgo: str, valor: str, **kwargs) -> None`:

    Este es el inicializador de la clase, y asigna los siguientes atributos:

`self.rasgo`	Indica qué parte del perro determina este gen. Por ejemplo `"pelo"`, `"orejas"`, `"ojos"`.
`self.valor`	El valor del rasgo determinado por el gen. Por ejemplo `"Azules"`, `"Verdes"`, `"Cafes"`.

    Además, el método `__init__` hace una llamada a `super().__init__(**kwargs)`. Esto facilita llamar a todos los métodos `__init__` en el caso de multiherencia de clases.
  - `def registrar(cls, subclase) -> None`:

    Método de clase[1] que permite agregar subclases de genes al atributo de clase `REGISTRO`. Este método se llama usando "`Gen.registrar(sub_clase)`".
  - `def __add__(self, other: Gen): -> ParDeGenes`:

    Es un método abstracto[2]. Define el comportamiento de la clase al usar el operador "+". Como es un método abstracto, este debe ser implementado en las sub-clases.
  - `def __repr__(self) -> str`:

    Retorna el `valor` del gen para representarlo al usar el método `print` o `str` en un gen.

[1] Para indicar que un método es "método de clase" se utiliza el decorador "`@classmethod`".
[2] Para indicar que un método es "abstracto" se utiliza el decorador "`@abstractmethod`".

- No modificar `class Mutable`:
  - `def __init__(self, **kwargs) -> None`:

    Este es el inicializador de la clase. Llama a `super().__init__(**kwargs)` para permitir multiherencia, y además asigna el siguiente atributo:

`self.probabilidades_mutar -> list[tuple[str, float]]`	Corresponde a las probabilidades de mutación del gen. Por ejemplo, si un gen de ojos azules tiene `1 %` de probabilidades de mutar a ojos Cafes, y `3 %` de probabilidades de mutar a ojos Verdes, el valor de `probabilidades_mutar` sería: `[("Cafes", 0.01), ("Verdes", 0.03), ("Azules", 0.96)]`

  - `def mutar_random(self) -> str`:

    Retorna un nuevo valor para el gen basado en las probabilidades de mutación. Utiliza la función `elegir_con_probabilidades`.

  - `def get_probabilidades_mutar(self) -> list[tuple[str, float]]`:

    Es un método abstracto[2]. Debe ser implementado por las subclases y define las probabilidades de mutación del gen.

- No modificar `class ParDeGenes`:
  - `def __init__(self, gen1: Gen, gen2: Gen, fenotipo: str) -> None`:

    Este es el inicializador de la clase, y asigna los siguientes atributos:

`self.par_de_genes`	Una tupla que contiene los dos genes del par.
`self.rasgo`	El rasgo asociado al par de genes, obtenido desde `gen1`. Por ejemplo: `"pelo"`
`self.fenotipo`	Representa cómo se manifiestan ambos genes del par. Esto depende de el tipo de dominancia de los genes.

  - `def elegir_alelo_aleatorio(self) -> Gen`:

    Selecciona uno de los dos genes del par de forma aleatoria. Si el gen seleccionado es una instancia de `Mutable`, puede mutar a un nuevo valor usando sus probabilidades de mutación.

  - `def __repr__(self) -> str`:

    Define la representación del objeto, retornando el `fenotipo`.

Parte 1. Dominancia

En esta parte de la actividad debes completar las clases definidas en el archivo "`dominancia.py`". Para esto, debes simplemente completar el método `__add__` en cada una de las clases.

En todas las clases de dominancia, el método `__add__(self, other)` debe determinar el `fenotipo` resultante al combinar los 2 genes, y debe retornar una instancia de la clase `ParDeGenes` con los genes `self` y `other`, y el `fenotipo` determinado.

A nivel de las características ya definidas, se aplican las siguientes reglas de dominancia para los distintos rasgos[3]:

- **Genes dominantes**: Para cada rasgo, existe a lo más 1 tipo de gen Dominante. Por ejemplo, el gen dominante de pelo es `PeloNegro`, y el gen dominante de ojos es `OjosAzules`. En el caso de las orejas, no hay gen dominante.

  Si se combina un gen dominante con cualquier otro gen, el fenotipo resultante es el valor del gen dominante. Por ejemplo, si se suma un gen de `OjosCafes` con un gen de `Ojosverdes`, el fenotipo final es `"Cafes"`.

- **Genes recesivos**: Para cada rasgo, existe a lo más 1 tipo de gen Recesivo. Por ejemplo, el gen recesivo de pelo es `PeloBlanco`, y en el caso de las orejas y los ojos, no hay gen recesivo.

  Si se combina un gen recesivo con otra copia del mismo gen, el fenotipo resultante es el valor del gen recesivo. Por ejemplo, si se suman dos copias del gen `PeloBlanco`, el fenotipo final es `"Blanco"`.

  En cualquier otro caso, el fenotipo resultante es el valor del otro gen.

- **Genes codominantes**: Muchos tipos de genes codominantes pueden existir para un mismo rasgo. Sin embargo, no pueden existir genes codominantes y recesivos en el mismo rasgo. En otras palabras, nunca se va a mezclar un gen codominante con un gen recesivo.

  Si se suma un gen codominante con un gen dominante, el fenotipo final es el valor del gen dominante.

  Al sumar un gen codominante con una copia del mismo gen, el fenotipo resultante es el valor del gen. Por ejemplo `OrejasLargas` + `OrejasLargas`, el fenotipo final es `"Largas"`.

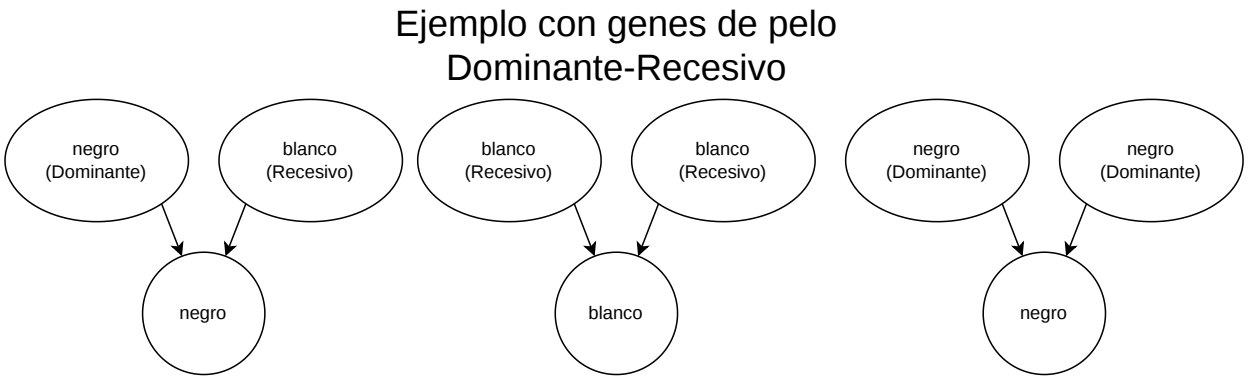
  Si ambos genes son codominantes pero tienen valores distintos, entonces el fenotipo es una mezcla entre ambos genes. En este caso el valor de la variable `fenotipo` debe ser:

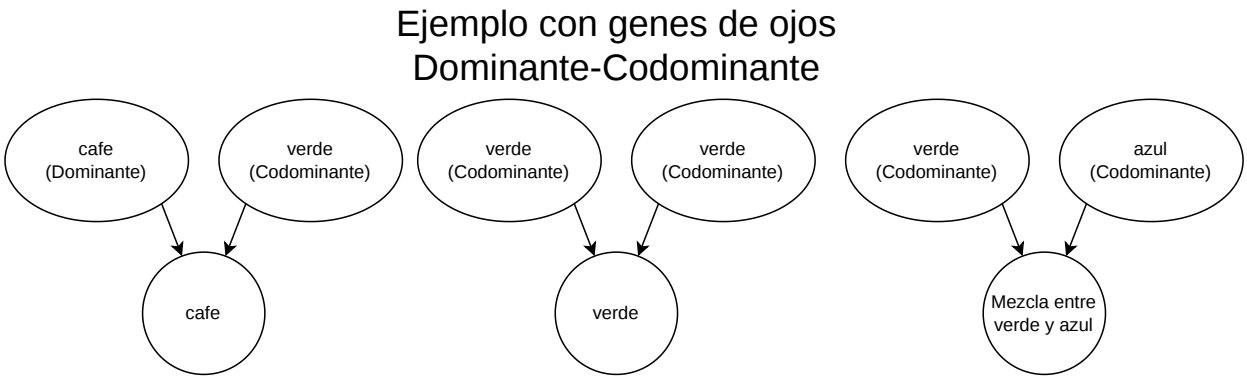
  `"Mezcla entre {valor_gen_1} y {valor_gen_2}"`

Puedes asumir que no existen casos en que se combinan genes de distintos rasgos, por ejemplo: `pelo` + `ojos`.

Es útil usar el método `isinstance()` para determinar si un gen es una instancia de `Recesivo`, `Dominante` o `Codominante`

[3] Las opciones de rasgos son: Pelo, Ojos y Orejas.





Parte 2. Mutabilidad

Gracias a tu ayuda en la Parte 1 se descubrió que el gen `OjosCafes` es un gen que puede mutar. En esta sección debes agregar el comportamiento de mutación al programa en el gen `OjosCafes` en el archivo "`caracteristicas.py`".

Para esto debes hacer que `OjosCafes` herede de `Dominante` y de `Mutable` a la vez. Como la clase `Mutable` tiene un método abstracto `get_probabilidades_mutar`, debes implementar el método `get_probabilidades_mutar` en `OjosCafes`. Las probabilidades de mutar del gen `OjosCafes` son:

```
[("Cafes", 0.7), ("Verdes", 0.1), ("Azules", 0.2)]
```

Notas

- **No puedes hacer *import* de otras librerías externas a las entregadas en el archivo.**
- La entrega se realiza subiendo los archivos `caracteristicas.py` y `dominancia.py` a la carpeta `Actividades/AC3` de tu repositorio personal. Recomendamos no subir otros archivos.
- Recuerda que la ubicación de tu entrega es en **tu repositorio de Git**. En la rama (*branch*) por defecto del repositorio: `main`.
- Se recomienda completar la actividad en el orden del enunciado.
- Recuerda que esta evaluación presenta corrección **automatizada**. Si entregas un código que se cae al momento de correr los *tests*, será evaluado con 0 puntos.
- Siéntete libre de agregar nuevos `print` en cualquier lugar para revisar objetos, modificaciones, etc... Es una herramienta muy útil para encontrar errores. Y si aparece un error inesperado, ¡léelo! Intenta interpretarlo.

Objetivos de la actividad

- Implementar clases siguiendo especificaciones técnicas.
- Manejar concepto de herencia, multiherencia y clases abstractas correctamente.
- Utilizar métodos de clases padres correctamente.
- Probar código mediante la ejecución de *test*.

Ejecución de *tests*

En esta actividad se provee de varios archivos `.py` los cuáles contiene diferentes *tests* que ayudan a validar el desarrollo de la actividad. Para ejecutar estos *tests*, **primero debes posicionar tu terminal/consola en la carpeta de la actividad (`Actividades/AC3`)**. Luego, desde esta misma, debes escribir el siguiente comando para ejecutar todos los *tests* de la actividad:

```
python3 -m unittest discover tests_publicos -v -b
```

Si quieres solo ejecutar los tests de la parte 1:

```
python3 -m unittest -v -b tests_publicos.tests_parte_1
```

Si quieres solo ejecutar los tests de la parte 2:

```
python3 -m unittest -v -b tests_publicos.tests_parte_2
```

En cambio, si deseas ejecutar un test en particular, puedes hacer usando el comando:

```
python3 -m unittest -v -b <carpeta>.<archivo>.<NombreClase>.<nombre_test>
```

Por ejemplo:

```
python3 -m unittest -v -b tests_publicos.tests_parte_1.VerificarAdd.test_dominante_dominante
```

Importante: recuerda que si `python3` no funciona, probar con el comando específico de tu computador. Este puede ser `py`, `python`, `py3` o `python3.12`.